Testing and DevOps

How this product is tested, shipped, run and recovered. The rule is simple: a video pipeline fails in expensive places - transcription, generation and render - so every gate below exists to catch failure before it burns credits or a user's deadline.

> **Stack this doc assumes:** Next.js frontend, FastAPI orchestrator, Redis + Celery queues, faster-whisper, Remotion, ffmpeg, R2 / S3 storage. Web app for Windows and Mac. Render is 1080p in MVP, 4K later. Pricing is credits / subscription because the product carries render costs.

Scope in one line

The 10-stage flow under test: topic → research → script → voice (recorded or AI) → voice cleanup → visual plan → generation into project → motion graphics → timeline edit → captions / sound → preview → render → publish pack (Shorts / Reels, thumbnails, title / description).

How to use this doc

- **Builders:** Sections 2-5 are the merge gate. Nothing merges without them.
- **On-call:** Sections 7-8 are the runbook. Start there during an incident.
- **Founder:** Section 4 (cost targets) and Section 9 (the one open decision) need your sign-off before the spike ends.

Clean-room product. Patterns borrowed from comparable editors; no code, text or designs copied from Slot Studio, OpenChatCut, Palmier Pro or Pallaidium. All third-party costs in this doc are estimates / ranges, not quotes. Version 1.0 - for the 5-7 day spike and 6-8 week MVP.

1. Quality gates - what "done" means

A stage is done when its tests pass, its cost is measured, and its failure mode is visible in monitoring. Feature-complete without those three is not done.

Gate	Pass condition
Unit + lint + types	CI green. No skipped tests on the changed module.
Integration	Stage contract tests pass against real Redis, Postgres and object storage (test containers, not mocks).
Golden render	Perceptual diff within threshold (Section 3). Audio loudness and caption timing within tolerance.
Cost check	Job cost recorded per stage. PRs that raise cost per finished minute by more than 10% need an explicit note and approval.
Security	Dependency scan clean at high severity. No secrets in repo. New external calls reviewed for data sent.
Accessibility (UI)	Keyboard path for timeline edit, visible focus, contrast pass on editor chrome. Captions are a product feature, not an afterthought.

Defect severity

- **S0 - Blocker:** data loss, wrong user charged, render publishes someone else's media, security breach. Stop the release.
- **S1 - Major:** a stage fails for many users, or output is visibly / audibly wrong. Fix before next release.
- **S2 - Minor:** workaround exists. Fix within two sprints.
- **S3 - Polish:** backlog, triaged weekly.

Test data rule

Use synthetic voices, licensed stock clips and scripts written for testing. Never test with a real user's recording, a scraped creator video, or a production database copy. YouTube-sourced clips appear in tests only for the opt-in path, using clips cleared for that test.

2. Test plan - by layer

Unit tests

- **Frontend (Vitest + Testing Library):** timeline operations (trim, split, reorder, undo), caption editor, credit-balance maths, upload validation.
- **Backend (pytest):** script / plan validators, credit ledger (integer minor units only, no floats), idempotency keys, retry policy, webhook signature checks.
- **Media helpers:** timestamp parsing, SRT / VTT generation, loudness normalisation targets, thumbnail safe-area maths.
- Target: 80% line coverage on ledger, timeline model and job state machine. Coverage elsewhere is reported, not chased.

Integration tests

Boundary	What is verified
FastAPI ↔ Celery ↔ worker	Job accepted, state transitions in order, result written, failure produces a user-readable error - not a stuck spinner.
faster-whisper	Known audio fixture transcribes within word-error tolerance; word timestamps stay monotonic; long-file chunking reassembles correctly.
Voice cleanup	Filler / breath removal never clips word starts on the fixture set; cleaned audio duration change is reported.
Generation providers	Contract tests per provider with recorded responses; live smoke test daily, not per PR, to control cost.
Storage (R2 / S3)	Signed upload / download, range reads for preview, lifecycle expiry of temp files, cross-user access denied.
Billing	Credits reserved at job start, settled on success, refunded on system failure, never double-charged on retry.

2 continued - End-to-end and contract tests

E2E journeys (Playwright, run on every staging deploy)

1. **Happy path, AI voice:** topic → script accepted → AI voice → visual plan → timeline → captions → preview → 1080p render → publish pack downloads. Assert final MP4 duration, resolution and audio track.
2. **Happy path, recorded voice:** upload fixture recording → cleanup → captions generated from cleaned audio → render. Assert captions match the cleaned, not raw, audio.
3. **Failure path:** kill a render worker mid-job. Assert the job retries or fails cleanly, credits are refunded, and the user sees a retry button - in that order.

Keep E2E to these three journeys plus login, billing and project-delete. Everything else belongs at integration level; a large E2E suite will be slow, flaky and ignored.

Contract and schema tests

- OpenAPI schema snapshot for FastAPI; breaking changes fail CI.
- Remotion composition props are versioned. Old projects must still render after a template change, or a migration runs first.
- Publish-pack manifest (files, sizes, hashes) is schema-checked so Shorts, thumbnails and metadata never drift apart.

Flakiness policy

An intermittently failing test is quarantined the same day, ticketed, and out of the gate for at most 5 working days. A second quarantine means it is rewritten, not re-run until green.

3. Golden render tests and media QA

Pixel-perfect comparison fails across GPUs and ffmpeg versions, so golden tests compare what a viewer perceives.

Check	Method and tolerance
Video frames	Sample frames at fixed timestamps. Perceptual hash + SSIM against golden; threshold set per template, reviewed when a template intentionally changes.
Audio	Integrated loudness within +/- 1 LUFS of target (-14 LUFS for publish pack); true peak at or below -1 dBTP; no silent gaps over 2s except scripted pauses.
Captions	Caption text equals transcript fixture; cue start / end within 120 ms; no cue longer than 7s or over 2 lines in the rendered burn-in check.
Motion graphics + thumbnails	Chart fixtures at 3 timestamps, data labels OCR-checked against source data; thumbnail dimensions, safe-area and mobile legibility checked per template release.

Goldens live in versioned storage with the template version in the path. Updating a golden requires a second reviewer and a note explaining the intentional visual change.

4. Performance, load and cost targets

Metric	Target (MVP)
API response (non-job endpoints)	p95 under 300 ms at expected launch load.
Job acceptance	p95 under 800 ms; user always gets a job ID or a clear rejection.
Queue wait, AI stages	p95 under 60s at normal load; visible queue position after 30s.
Transcription speed	Faster than 0.3x real time on standard worker (10-min audio in under 3 min).
Preview render	Low-res preview ready in under 30s for a 3-min project.
Final render, 1080p	6-min video in under 15 min on standard render worker. Measured and published internally per template.
Concurrent renders	System sustains the paid-plan concurrency sold; load test at 2x that number before launch.
Error rate	Under 1% of jobs fail for system reasons, excluding user-input failures.

Load testing

- k6 (or equivalent) scripts for API + job submission, run weekly against staging with synthetic media jobs.
- A monthly render soak: mixed project lengths and templates for 2 hours. Watch queue depth, worker memory, temp-disk growth and cost per minute.
- Never load-test a third-party generation provider beyond its rate limits; mock it and test our back-off separately.

Cost targets (estimates)

Track **cost per finished minute** as the headline number, split by transcription, generation, render compute and storage / egress. Alert when the 7-day average rises more than 15%. Infrastructure at MVP scale is estimated at a few hundred US dollars per month before generation-provider spend (estimate / range, varies with render volume); generation and AI voice are the variable cost and must be metered per job, per user. Credits are priced from this measured cost, never guessed.

5. Environments and CI / CD

Environment	Purpose and rules
Local	Docker Compose: Next.js, FastAPI, Postgres, Redis, MinIO (S3-compatible). One command up. Seed data is synthetic only.
CI	Ephemeral per PR. Unit, integration (test containers), lint, types, dependency scan, OpenAPI snapshot.
Staging	Mirrors production topology at smaller size. Real providers in sandbox / low-cost mode. E2E + golden suite run here. All merges deploy here automatically.
Production	Promoted from the exact staging artefact - same image digests, never a rebuild. Manual approval, weekday window, rollback ready.

Pipeline (per pull request)

1. Lint, types, unit tests (frontend + backend).
2. Integration tests with test containers.
3. Build images; scan images and dependencies.
4. On merge to main: deploy to staging, run E2E + golden renders, publish cost report for the fixture project.
5. On release tag: promote to production, run smoke checks (login, create project, 30-second test render), watch dashboards for 30 minutes.

Branching and releases

- Trunk-based: short-lived branches, merge within 2 days, feature flags for anything unfinished.
- Database migrations are expand-then-contract, so the previous release can still run during rollback.
- Remotion templates and worker images are versioned together; a project records the template version it was rendered with.
- Secrets live in the platform secret store only. CI uses short-lived credentials. No secret is ever baked into an image or a Remotion bundle.

6. Infrastructure - queues, workers, storage

Queue design (Redis + Celery)

Queue	Work	Scaling rule
`api-jobs`	Research, script, visual plan - short, CPU-light.	Scale on queue depth; high concurrency per worker.
`ai-jobs`	Transcription (faster-whisper), voice cleanup, caption timing.	Scale on queue depth + job age; GPU worker optional later, CPU baseline must pass targets first.
`media-jobs`	Generation into project, stock download, thumbnail and publish-pack assembly.	Scale on depth; strict per-provider rate limiting and back-off.
`render-jobs`	Remotion render + ffmpeg encode / mux.	Scale on depth with a hard concurrency cap tied to cost. One heavy render per worker.

- Every job carries `project_id`, `job_id`, `idempotency_key` and a deadline. Jobs past deadline fail visibly instead of running forever.
- Retries: maximum 3, exponential back-off, only for transient failures. Bad input, insufficient credits and provider content rejections never retry.
- Dead-letter queue for exhausted jobs, reviewed daily. A dead-letter spike is an incident signal, not background noise.

Storage (R2 / S3)

- Buckets separated by purpose: uploads (private), project assets (private), renders (private, signed delivery), publish packs (private, signed delivery), goldens (versioned, no public access).
- Temp and preview files expire by lifecycle rule (7 days). Final renders follow the plan's retention promise and are restorable (Section 7).
- All uploads scanned for type and size before any worker touches them. Filenames are system-generated; user filenames are display metadata only.

Configuration

One config schema per service, validated at boot. A worker with invalid config refuses to start rather than running with silent defaults. Provider keys are per-environment, so staging can never spend production credits unnoticed.

7. Observability, backups and operational security

Observability

- **Logs:** structured JSON, keyed by `request_id`, `job_id`, `project_id`. No user audio, transcripts or provider keys in logs.
- **Metrics:** queue depth and job age per queue, stage duration, render time per template, cost per job, provider error rate, credit reserve / settle / refund counts.
- **Traces:** one trace per job across FastAPI, Celery and workers, so a slow video can be followed stage by stage.
- **Dashboards:** one operations view (health, queues, cost) and one product view (jobs started / finished, stage drop-off). If it is not on one of these, it is not monitored.
- **Alerts:** page on S0 / S1 signals only - error rate, dead-letter spike, render success below threshold, payment webhook failures, disk / storage near full. Everything else is a next-day ticket. Alert fatigue is an outage in slow motion.

Backups and restore

Asset	Backup	Restore test
Postgres (projects, ledger, users)	Automated daily backup + point-in-time recovery where the platform supports it; 30-day retention.	Monthly restore into staging; ledger totals must reconcile exactly.
Object storage (assets, renders)	Versioning on asset and render buckets; deletion is soft-delete first.	Quarterly sample restore of a full project, including publish pack.
Config and infra definitions	In version control; secrets referenced, never stored.	Staging rebuild from repo once per release cycle.

Operational security

- Least-privilege access; production access is named, logged and reviewed monthly. No shared root accounts.
- Dependency and image scanning in CI; critical patches within 48 hours.
- Rate limits on auth, upload and job creation. Abuse of the opt-in YouTube path is blocked at the feature flag, not by policy text alone.

8. Incident runbook and release operations

Severity and response

Severity	Example	Response
SEV-1	Site down, billing wrong, data exposure, renders delivering wrong user's media.	Respond immediately, stop deploys, founder notified, status update every 30 min.
SEV-2	One stage failing for many users, render backlog growing, provider outage.	Respond within 1 hour in working hours, status update every 2 hours.
SEV-3	Single-template defect, slow previews, non-critical provider degradation.	Ticket same day, fix in normal flow.

First steps, by failure class

- **Queue backlog:** check job age and dead-letter count; confirm workers are alive before scaling; scale the affected queue only; if a provider is the bottleneck, pause new jobs for that stage with a clear user message.
- **Render failures:** pull one failing `job_id` trace end to end; check template version, worker disk and ffmpeg / Remotion logs; roll back the template version before debugging forward.
- **Billing mismatch:** freeze credit settlement for affected jobs, reconcile reserve / settle / refund from the ledger, refund first and investigate second. Never adjust balances by hand without a ledger entry.
- **Provider outage:** switch the stage to its fallback if one exists, or fail jobs cleanly with refunds. Do not queue silently for hours.
- **Bad deploy:** roll back to the previous image digests (target: under 10 minutes). Migrations must already be rollback-safe (Section 5).

After every SEV-1 / SEV-2

Short written review within 3 working days: what happened, detection time, user impact, credit impact, and one concrete prevention item with an owner and a date. No blame, no generic lessons.

9. The one open decision

> **Open decision - render worker hosting.** Options: **(A)** Managed cloud VMs / containers we operate - full control of Remotion + ffmpeg, we own scaling and patching. **(B)** A serverless / managed render service - less operations, less control over codecs, templates and per-render cost. **Recommendation: A for MVP,** because render cost and template control are the business. Revisit B only when render volume makes worker operations the bottleneck. Decide before the spike ends; the CI and queue design above works with either, but worker images and cost targets do not.

Everything else in this doc is decided for the MVP. Changing a target above means changing the doc in the same pull request as the code.